leetcode 2

72将一个字符转换为另一个的最小操作次数

C

```c
int minDistance(char* word1, char* word2) {
    int m=strlen(word1),n=strlen(word2);
    int dp[m+1][n+1];
    for (int i=0;i<=m;i++) {
        dp[i][0]=i;
    }
    for (int i=0;i<=n;i++) {
        dp[0][i]=i;
    }
    for (int i=1;i<=m;i++) {
        for (int j=1;j<=n;j++) {
            if (word1[i-1]==word2[j-1]) dp[i][j]=dp[i-1][j-1];
            else {
                int min=dp[i-1][j-1];
                if (dp[i-1][j]<min) min=dp[i-1][j];
                if (dp[i][j-1]<min) min=dp[i][j-1];
                dp[i][j]=min+1;
            }
        }
    }
    return dp[m][n];
}
```



中等 106,771 提交 79,528 通过

给定两个字符串 s1 和 s2，返回 *使两个字符串相等所需删除字符的 ASCII 值的最小和* 。

示例 1:

输入： s1 = "sea", s2 = "eat"
输出： 231
解释： 在 "sea" 中删除 "s" 并将 "s" 的值(115)加入总和。
在 "eat" 中删除 "t" 并将 116 加入总和。
结束时，两个字符串相等，115 + 116 = 231 就是符合条件的最小和。

示例 2:

输入： s1 = "delete", s2 = "leet"
输出： 403
解释： 在 "delete" 中删除 "dee" 字符串变成 "let"，
将 100[d]+101[e]+101[e] 加入总和。在 "leet" 中删除 "e" 将 101[e] 加入总和。
结束时，两个字符串都等于 "let"，结果即为 100+101+101+101 = 403 。
如果改为将两个字符串转换为 "lee" 或 "eet"，我们会得到 433 或 417 的结果，比答案更大。

提示:

- 1 <= s1.length, s2.length <= 1000
- s1 和 s2 由小写英文字母组成

中等 106,768 提交 79,525 通过

给定两个字符串 s1 和 s2，返回 *使两个字符串相等所需删除的 ASCII 值的最小和* 。

示例 1:

输入： s1 = "sea", s2 = "eat"
输出： 231
解释： 在 "sea" 中删除 "s" 并将 "s" 的值(115)加
在 "eat" 中删除 "t" 并将 116 加入总和。
结束时，两个字符串相等，115 + 116 = 231 就是符合
小和。

示例 2:

输入： s1 = "delete", s2 = "leet"
输出： 403
解释： 在 "delete" 中删除 "dee" 字符串变成 "let
将 100[d]+101[e]+101[e] 加入总和。在 "leet" 中
"e" 将 101[e] 加入总和。
结束时，两个字符串都等于 "let"，结果即为
100+101+101+101 = 403 。
如果改为将两个字符串转换为 "lee" 或 "eet"，我们会
433 或 417 的结果，比答案更大。

思路

1. 动态规划
2. 定义 `dp[i][j]`
   21. `dp[i][j]` 代表 `word1` 中前 `i` 个字符，变换到 `word2` 中前 `j` 个字符，最短需要操作的次数
   22. 需要考虑 `word1` 或 `word2` 一个字母都没有，即全增加/删除的情况，所以预留 `dp[0][j]` 和 `dp[i][0]`
3. 状态转移
   31. **增**，`dp[i][j] = dp[i][j - 1] + 1`
   32. **删**，`dp[i][j] = dp[i - 1][j] + 1`
   33. **改**，`dp[i][j] = dp[i - 1][j - 1] + 1`
   34. 按顺序计算，当计算 `dp[i][j]` 时，`dp[i - 1][j]`，`dp[i][j - 1]`，`dp[i - 1][j - 1]` 均已经确定了
   35. 配合增删改这三种操作，需要对应的 `dp` 把操作次数加一，取三种的最小
   36. 如果刚好这两个字母相同 `word1[i - 1] = word2[j - 1]`，那么可以直接参考 `dp[i - 1][j - 1]`，操作不用加一

712使字符串相等的最小ASCII码和

C

```c
int minimumDeleteSum(char* s1, char* s2) {
    int m=strlen(s1),n=strlen(s2);
    int dp[m+1][n+1];
    dp[0][0]=0;//直接加就行
    for (int i=1;i<=m;i++) dp[i][0]=dp[i-1][0]+s1[i-1];
    for (int j=1;j<=n;j++) dp[0][j]=dp[0][j-1]+s1[j-1];
    for (int i=1;i<m;i++) {
        for (int j=1;j<n;j++) {
            if (s1[i-1]==s2[j-1]) dp[i][j]=dp[i-1][j-1];
            else {
                int min=dp[i-1][j-1]+s1[i-1]+s2[j-1];
                if (dp[i-1][j]+s1[i-1]<min) min=dp[i-1][j]+s1[i-1];
                if (dp[i][j-1]+s2[j-1]<min) min=dp[i][j-1]+s2[j-1];
            }
        }
    }
    return dp[m][n];
}
```

63不同路径II

Plain Text

```
给定一个 m x n 的整数数组 grid。一个机器人初始位于 左上角（即 grid[0][0]）。机器人尝试移动到 右下角（即 grid[m - 1][n - 1]）。机器人每次只能向下或者向右移动一步。
网格中的障碍物和空位置分别用 1 和 0 来表示。机器人的移动路径中不能包含 任何 有障碍物的方格。
返回机器人能够到达右下角的不同路径数量。
测试用例保证答案小于等于 2 * 109。
```

C

```c
int uniquePathsWithObstacles(int** obstacleGrid, int obstacleGridSize, int* obstacleGridColSize) {
    int row = obstacleGridSize, col = obstacleGridColSize[0];
    int ans[row][col];
    ans[0][0] = (obstacleGrid[0][0] == 0) ? 1 : 0;// 初始化起点：如果是障碍物则为0，否则为1
    for (int i = 0; i < row; i++) {
        for (int j = 0; j < col; j++) {
            if (i == 0 && j == 0) continue;// 起点已初始化，跳过
            if (obstacleGrid[i][j] == 1) {
                ans[i][j] = 0;
                continue;
            }// 如果当前格子是障碍物，路径数为0
            // 计算从上方的路径数（如果上方存在且不是障碍物）
            int fromTop = (i - 1 >= 0 && obstacleGrid[i-1][j] == 0) ? ans[i-1][j] : 0;
            // 计算从左方的路径数（如果左方存在且不是障碍物）
            int fromLeft = (j - 1 >= 0 && obstacleGrid[i][j-1] == 0) ? ans[i][j-1] : 0;
            ans[i][j] = fromTop + fromLeft;
        }
    }
    return ans[row-1][col-1];
}
```

221最大正方形面积

C

```c
/*
 *在一个由 '0' 和 '1' 组成的二维矩阵内，找到只包含 '1' 的最大正方形，并返回其面积。
 *思路：定义的矩阵每个位置保存的是以该位置为右下角的正方形的边长
 *如果这个位置是0那就是0
 *如果这个位置在两边或上左左上三个位置中有一个为'0'则为1
 *如果周围都是1那么这个位置的值就是那三个中最小的一个再加1
 *注意是char类型输入，输出的是面积不是边长1
 */
int maximalSquare(char** matrix, int matrixSize, int* matrixColSize) {
    int row=matrixSize,col=matrixColSize[0],result=0;
    int ans[row][col];
    for (int i=0;i<row;i++) {
        for (int j=0;j<col;j++) {
            if (matrix[i][j]=='0') ans[i][j]=0;
```

```
            else if (i==0||j==0||matrix[i-1][j]=='0'||matrix[i][j-1]=='0'||matrix[i-1][j-1]=='0') ans[i][j]=1;
            else {
                int temp=ans[i-1][j-1];
                temp=temp<ans[i-1][j]?temp:ans[i-1][j];
                temp=temp<ans[i][j-1]?temp:ans[i][j-1];
                ans[i][j]=temp+1;
            }
            result=ans[i][j]>result?ans[i][j]:result;
        }
    }
    return result*result;
}
```

931下降路径最小和

C

```
//给你一个 n x n 的 方形 整数数组 matrix ，请你找出并返回通过 matrix 的下降路径 的最小和 。下降路径 可以从第一行中的任何元素开始，并从每一行中选择一个元素。在下一行选择的元素和当前行所选元素最多相隔一列（即位于正下方或者沿对角线向左或者向右的第一个元素）。具体来说，位置 (row, col) 的下一个元素应当是 (row + 1, col - 1)、(row + 1, col) 或者 (row + 1, col + 1) 。
int minFallingPathSum(int** matrix, int matrixSize, int* matrixColSize) {
    if (matrixSize==1) return matrix[0][0];
    int row=matrixSize,col=matrixColSize[0],result=INT_MAX;
    int ans[row][col];
    for (int i=0;i<row;i++) {
        for (int j=0;j<col;j++) {
            ans[i][j]=matrix[i][j];
            if (i==0) continue;
            else {
                if (j==0) ans[i][j]+=ans[i-1][j]<ans[i-1][j+1]?ans[i-1][j]:ans[i-1][j+1];
                else if (j==col-1) ans[i][j]+=ans[i-1][j-1]<ans[i-1][j]?ans[i-1][j-1]:ans[i-1][j];
                else {
                    int temp=ans[i-1][j];
                    temp=temp<ans[i-1][j-1]?temp:ans[i-1][j-1];
                    temp=temp<ans[i-1][j+1]?temp:ans[i-1][j+1];
                    ans[i][j]+=temp;
                }
            }
        }
    }
```

```
    for (int i=0;i<col;i++) result=ans[row-1][i]<result?ans[row-1][i]:result;
    return result;
}
```

1143最长公共子序列

C

```c
//给定两个字符串 text1 和 text2，返回这两个字符串的最长 公共子序列 的长度。如果不存在 公共子序列 ，返回 0 。  一个字符串的 子序列 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。  例如，"ace" 是 "abcde" 的子序列，但 "aec" 不是 "abcde" 的子序列。  两个字符串的 公共子序列 是这两个字符串所共同拥有的子序列。
int longestCommonSubsequence(char* text1, char* text2) {
    int len1 = strlen(text1), len2 = strlen(text2);

    // 创建动态规划表，大小为 (len1+1) x (len2+1)
    int dp[len1+1][len2+1];

    // 初始化第一行和第一列为0
    for (int i = 0; i <= len1; i++) dp[i][0] = 0;
    for (int j = 0; j <= len2; j++) dp[0][j] = 0;

    // 填充动态规划表
    for (int i = 1; i <= len1; i++) {
        for (int j = 1; j <= len2; j++) {
            if (text1[i-1] == text2[j-1]) {
                // 字符匹配，长度加1
                dp[i][j] = dp[i-1][j-1] + 1;
            } else {
                // 字符不匹配，取上方或左方的最大值
                dp[i][j] = dp[i-1][j] > dp[i][j-1] ? dp[i-1][j] : dp[i][j-1];
            }
        }
    }

    return dp[len1][len2];
}
```

673对一个未排序数组，找出其最长递增序列的长度

C

```c
int findNumberOfLIS(int* nums, int numsSize) {
    if (numsSize == 0) return 0;
    int ans[2][numsSize];  // ans[0][i]: 以nums[i]结尾的最长递增子序列长度, ans[1][i]: 以nums[i]结尾的最长递增子序列个数
    int maxLen = 0;        // 全局最长递增子序列长度
    int result = 0;        // 最终结果：最长递增子序列的个数
    for (int i = 0; i < numsSize; i++) {
        ans[0][i] = 1;     // 至少包含自己，长度为1
        ans[1][i] = 1;     // 至少包含自己，个数为1
        // 查找以nums[i]结尾的最长递增子序列
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {// 如果找到更长的递增子序列
                if (ans[0][j] + 1 > ans[0][i]) {
                    ans[0][i] = ans[0][j] + 1;
                    ans[1][i] = ans[1][j];  // 重置计数
                }// 如果找到相同长度的递增子序列
                else if (ans[0][j] + 1 == ans[0][i]) {
                    ans[1][i] += ans[1][j];  // 累加计数
                }
            }
        }// 更新全局最长长度
        if (ans[0][i] > maxLen) {
            maxLen = ans[0][i];
            result = ans[1][i];  // 重置结果计数
        }// 如果找到相同长度的
        else if (ans[0][i] == maxLen) {
            result += ans[1][i];  // 累加结果计数
        }
    }
    return result;
}
```

279将一个数分为完全平方数和的最小个数

C

```c
int numSquares(int n) {
    int ans[n+1];
    ans[0] = 0;
    for (int i = 1; i <= n; i++) {  // 修正1：循环条件改为 i<=n
        int sqrt_i = (int)sqrt(i);
        if (sqrt_i * sqrt_i == i) {  // 修正3：避免浮点数精度问题
            ans[i] = 1;  // 完全平方数直接是1
```

```
        } else {
            int min = i;  // 最大值为i个1相加
            // 修正2：遍历所有可能的完全平方数
            for (int j = 1; j <= sqrt_i; j++) {
                int square = j * j;
                if (ans[i - square] + 1 < min) {
                    min = ans[i - square] + 1;
                }
            }
            ans[i] = min;
        }
    }
    return ans[n];
}
```

C

```
//518 兑换零钱2
int cmp(const void* a, const void* b) {return *(int*)a-*(int*)b;}
int change(int amount, int* coins, int coinsSize) {
    int ans[amount+1];
    ans[0]=1;
    qsort(coins, coinsSize, sizeof (int), cmp);
    for (int i=1;i<amount+1;i++) {
        if (i<coins[0]) ans[i]=0;
        else {
            int count=0;
            for (int j=0;j<coinsSize;j++) {
                if (coins[j]>i) break;
                else if (ans[i-coins[j]]!=0) count+=ans[i-coins[j]];
            }
            ans[i]=count;
        }
    }
    return ans[amount];
}
//注意，上方的是排列数，下方的时组合数，下面的才是正确的
int cmp(const void* a, const void* b) {return *(int*)a - *(int*)b;}
int change(int amount, int* coins, int coinsSize) {
    unsigned long long ans[amount + 1];// 初始化
    for (int i = 0; i <= amount; i++) ans[i] = i==0?1:0;
    qsort(coins, coinsSize, sizeof(int), cmp);
    // 注意：这里硬币循环要在外层，金额循环在内层
    // 这样可以避免重复计数（确保是组合数而不是排列数）
    for (int j = 0; j < coinsSize; j++) {
        for (int i = coins[j]; i <= amount; i++) {
```

```
            ans[i] += (long long) ans[i - coins[j]];
        }
    }
    return ans[amount];
}
```

377组合总数

C

```
int cmp(const void* a, const void* b) {return *(int*)a-*(int*)b;}
int combinationSum4(int* nums, int numsSize, int target) {
    long long ans[target+1];
    ans[0]=1;
    qsort(nums, numsSize, sizeof (int), cmp);
    for (int i=1;i<=target;i++) {
        if (i<nums[0]) ans[i]=0;
        else {
            int count=0;
            for (int j=0;j<numsSize;j++) {
                if (nums[j]>i) break;
                else count+=ans[i-nums[j]]==0?0:ans[i-nums[j]];
            }
            ans[i]=count;
        }
    }
    return ans[target];
}//这个就是组合数!
```

2260相同牌之间的最小间距

C

```
int minimumCardPickup(int* cards, int cardsSize) {
    int min=INT_MAX;
    int pos[1000001];//用来记录最近一次出现的位置
    for (int i=0;i<1000001;i++) pos[i]=-1;//对每一个都初始化，避免随即结果
    for (int i=0;i<cardsSize;i++) {
        if (pos[cards[i]]==-1) pos[cards[i]]=i;//如果是此前没有出现过，则进行记录
        else {//此时说明之前有出现过
            min=i-pos[cards[i]]+1<min?i-pos[cards[i]]+1:min;
            pos[cards[i]]=i;
        }
```

```
    }//区分i和cards[i]
    return min==INT_MAX?-1:min;
}
```

2815数组中最大数相等的两数和值的最大值

C

```c
int maxSum(int* nums, int numsSize) {
    int dig[numsSize];
    for (int i = 0; i < numsSize; i++) {
        int maxDigit = 0, n = nums[i];  // 重命名局部变量
        while (n != 0) {  // 修复：处理 n=0 的情况
            int digit = n % 10;
            maxDigit = digit > maxDigit ? digit : maxDigit;
            n /= 10;
        }
        dig[i] = maxDigit;
    }
    int max[10][2] = {{0}}, result = 0;  // 保持二维数组名为 max
    for (int i = 0; i < numsSize; i++) {
        int digit = dig[i];
        int current = nums[i];
        if (current > max[digit][0]) {
            max[digit][1] = max[digit][0];
            max[digit][0] = current;
        }
        else if (current > max[digit][1]) max[digit][1] = current;
    }
    for (int i = 0; i < 10; i++) {
        if (max[i][0] != 0 && max[i][1] != 0) {
            int sum = max[i][0] + max[i][1];
            if (sum > result) result = sum;
        }
    }
    return result == 0 ? -1 : result;
}
```

3805对字符串组统计其中可以凯撒加密实现转变的对数

C

```c
long long countPairs(char** words, int wordsSize) {
    long long ans[wordsSize],result=0;
```

```
    int len=strlen(words[0]);
    for (int i=0;i<wordsSize;i++) {
        ans[i]=0;
    }
    for (int i=0;i<wordsSize;i++) {
        for (int j=i;j>=0;j--) {
            if (j!=i) {
                int dif=words[i][0]-words[j][0];
                for (int k=0;k<len;k++) {
                    if (words[i][k]!=words[j][k]+dif) break;
                    else ans[i]+=ans[j]=1;
                }
            }
        }
    }
    for (int i=0;i<wordsSize;i++) result+=ans[i];
    return result;
}
```

3371数组最大异常值

C

```
int cmp(const void* a,const void* b) {return *(int*)a-*(int*)b;}
int getLargestOutlier(int* nums, int numsSize) {
    qsort(nums, numsSize, sizeof (int), cmp);//从小到大进行排序
    long long sum=0;//记录所有数字的和
    int result;
    for (int i=0;i<numsSize; i++) sum+=nums[i];
    for (int i=numsSize-1;i>=0;i--) {//从大到小，确保是最大的异常值
        long long temp=sum-nums[i];//此处为出这个数以外所有数的和
        int src=temp%2==0?temp/2:INT_MAX;//按照题目，必有一个数等于这个和的一半，二分查找
        if (src!=INT_MAX) {//注意排除不合理选项的方法
            int left=0,right=numsSize,find=0;
            while (left<right) {
                int mid=left+(right-left)/2;
                if (nums[mid]<src) left=mid+1;
                else right=mid;
            }//找出的left应该是符合要求的，现在处理不符合的情况
            if (left!=i && nums[left]==src) find=1;
            else {
                if (left==i && i==0 && nums[1]==src) find=1;
                else if (left==i && i==numsSize-1 && nums[numsSize-2]==src) find=1;
```

```
                else if (left==i && i!=0 && i!=numsSize-1 && (nums[i-1]==src || nums[i+1]==src)) find=1;
            }//注意这里究竟排除了什么！这里实际上并没有排除所有i取极值的情况
            if (find==1) {
                result=nums[i];
                break;
            }
        }
    }
    return result;
}
```

1014数组中values[i]+values[j]+i-j的最大值

C

```
int maxScoreSightseeingPair(int* values, int valuesSize) {
    int ans=0,max=values[0];
    for (int j=1;j<valuesSize;j++) {
        ans=ans>max+values[j]-j?ans:max+values[j]-j;
        max=max>values[j]+j?max:values[j]+j;
    }
    return ans;
}
//不断枚举右边的j，同时维护左侧i的值。通过将计算方式拆解为i和j方便在每一次枚举时都不断更新，而不需要确定两个值（这样几乎必然进入n^2）
```

1814“好对子”数目

C

```
int countNicePairs(int* nums, int numsSize) {
    int use[numsSize];
    long long cnt[numsSize];
    //对每一个数，将式子转化为其与自己转置的差值。得到use数组中的值
    for (int i=0;i<numsSize;i++) {
        int temp=0,n=nums[i];
        while (n!=0) {
            temp=(n%10)+temp*10;
            n/=10;
        }
        use[i]=nums[i]-temp;
    }
    cnt[0]=0;//只要找出一个数之前的数中与之匹配的数字个数即可
```

```
    for (int i=0;i<numsSize;i++) {
        cnt[i]=0;
        for (int j=i;j>=0;j--) {
            if (use[j]==use[i] && i!=j) {
                cnt[i]+=cnt[j]+1;//若与这一个可以匹配则可以与前面每一个匹配，那么在此基础上+1
                break;
            }
        }
    }
    int result=0;
    for (int i=0;i<numsSize;i++) {
        result=(result+cnt[i])%1000000007;
    }
    return result;
}
//给你一个数组 nums ，数组中只包含非负整数。定义 rev(x) 的值为将整数 x 各个数字位反转得到的结果。比方说 rev(123) = 321 ， rev(120) = 21 。我们称满足下面条件的下标对 (i, j) 是 好的 ：0 <= i < j < nums.length  nums[i] + rev(nums[j]) == nums[j] + rev(nums[i])  请你返回好下标对的数目。由于结果可能会很大，请将结果对 109 + 7 取余 后返回。
```

枚举右，维护左→一次遍历！！！

2871有序三元组最大值

C

```
//给你一个下标从 0 开始的整数数组 nums。请你从所有满足 i < j < k 的下标三元组 (i, j, k) 中，找出并返回下标三元组的最大值。如果所有满足条件的三元组的值都是负数，则返回 0 。下标三元组 (i, j, k) 的值等于 (nums[i] - nums[j]) * nums[k] 。
long long maximumTripletValue(int* nums, int numsSize) {
    int maxi=0,maxsub=0;
    long long ans=0;
    for (int k=0;k<numsSize;k++) {
        ans=ans>(long long)maxsub*nums[k]?ans:(long long)maxsub*nums[k];
        maxsub=maxsub>maxi-nums[k]?maxsub:maxi-nums[k];
        maxi=maxi>nums[k]?maxi:nums[k];
    }
    return ans;
}
//本题本质还是枚举右维护左，但是是扩展到三元情况下的
//在这个题中，nums[i]，两数之差，三元组值这三个数需要在递推中不断更新，同时要注意更新顺序
//在以最右边乘数为枚举对象的同时，不断维护前段两个值
```

1497判断数组是否可以被分为能够被k整除的n/2组

C

```c
bool canArrange(int* arr, int arrSize, int k) {
    int cnt[k];
    for (int i=0;i<k;i++) cnt[i]=0;
    for (int i=0;i<arrSize;i++) cnt[(arr[i]%k+k)%k]++;
    if (cnt[0]%2!=0) return false;
    for (int i=1;i<k;i++) {
        if (cnt[i]!=cnt[k-i]) return false;
    }
    return true;
}
```

1031两个连续定长数组和最大值

C

```c
//给你一个整数数组 nums 和两个整数 firstLen 和 secondLen，请你找出并返回两个无重叠子数组 中元素的最大和，长度分别为 firstLen 和 secondLen 。长度为 firstLen 的子数组可以出现在长为 secondLen 的子数组之前或之后，但二者必须是无重叠。
int maxSumTwoNoOverlap(int* nums, int numsSize, int firstLen, int secondLen) {
    int firstsum[numsSize],secondsum[numsSize];
    int sumfs=0,sumsf=0,fsleft=0,sfleft=0;
    firstsum[0]=secondsum[0]=nums[0];
    for (int i=1;i<numsSize;i++) {//问题1:处理0、1位置数据
        if (i<firstLen) firstsum[i]=firstsum[i-1]+nums[i];
        else firstsum[i]=firstsum[i-1]+nums[i]-nums[i-firstLen];
        if (i<secondLen) secondsum[i]=secondsum[i-1]+nums[i];
        else secondsum[i]=secondsum[i-1]+nums[i]-nums[i-secondLen];
    }
    for (int i=0;i<numsSize;i++) {//问题2:更新顺序
        if (i>=secondLen+firstLen-1) fsleft=fsleft>firstsum[i-secondLen]?fsleft:firstsum[i-secondLen];//问题3、4:if条件中i的取值；firstsum中i的取值
        sumfs=sumfs>fsleft+secondsum[i]?sumfs:fsleft+secondsum[i];
        if (i>=firstLen+secondLen-1) sfleft=sfleft>secondsum[i-firstLen]?sfleft:secondsum[i-firstLen];
        sumsf=sumsf>sfleft+firstsum[i]?sumsf:sfleft+firstsum[i];
    }//注意更新顺序！！！
    return sumfs>sumsf?sumfs:sumsf;
}
```

2909和最小的山形三元组

C

```c
//给你一个下标从 0 开始的整数数组 nums 。如果下标三元组 (i, j, k) 满足下述全部条件，则认为它是一个 山形三元组 ：i < j < k；nums[i] < nums[j] 且 nums[k] < nums[j] 请你找出 nums 中 元素和最小 的山形三元组，并返回其 元素和 。如果不存在满足条件的三元组，返回 -1 。
int minimumSum(int* nums, int numsSize) {
    int minl[numsSize], minr[numsSize], result=INT_MAX;
    minl[0]=minr[numsSize-1]=INT_MAX;
    for (int i=1;i<numsSize;i++) {
        minl[i]=minl[i-1]<nums[i-1]?minl[i-1]:nums[i-1];
        minr[numsSize-1-i]=minr[numsSize-i]<nums[numsSize-i]?minr[numsSize-i]:nums[numsSize-i];
    }
    for (int i=1;i<numsSize-1;i++) {
        if (nums[i]>minl[i] && nums[i]>minr[i]) {
            result=result<nums[i]+minl[i]+minr[i]?result:nums[i]+minl[i]+minr[i];
        }
    }
    return result==INT_MAX?-1:result;
}//记录每个位置处左右两边的最小值并在确保一个数小于其两侧的最小值的情况下遍历，依次求和并更新
```

456编号ijk而数值从小到大ikj的模式存在（栈！！！）

C

```c
/* 解法2, 单调栈解法 */
bool find132pattern(int* nums, int numsSize){
    if (numsSize < 3) {
        return false;
    }
    int min[numsSize];
    int stack[numsSize];
    int top = 0;
    int i, j, num;

    /* 维护[0,j-1]区间的最小值, 即题目要求的1 */
    min[0] = nums[0];
    for (i = 1; i < numsSize; i++) {
        min[i] = fmin(nums[i], min[i - 1]);
    }
```

```
16
17      /* 使用单调栈去找3, 单调递减栈, 小数出栈 */
18      for (j = numsSize - 1; j >= 1; j--) {
19          num = INT_MIN;
20          while (top > 0 && nums[j] > stack[top - 1]) {
21              num = stack[--top];  /* 出栈元素3为小于a[j]的最大值 */
22          }
23          if (num > min[j]) {      /* 如果3大于1, 则返回true */
24              return true;
25          }
26          stack[top++] = nums[j];
27      }
28      return false;
29  }
```

2559范围内以元音首尾的字符串数量

C

```
1  /**
2   * Note: The returned array must be malloced, assume caller calls free().
3   */
4  int* vowelStrings(char** words, int wordsSize, int** queries, int
   queriesSize, int* queriesColSize, int* returnSize) {
5      int cnt[wordsSize+1];
6      cnt[0]=0;//转化为前缀之差
7      for (int i=1;i<=wordsSize;i++) {
8          int len=strlen(words[i-1]);
9          cnt[i]=cnt[i-1];
10         if ((words[i-1][0]=='a' || words[i-1][0]=='e' || words[i-1]
   [0]=='i' || words[i-1][0]=='o' || words[i-1][0]=='u') && (words[i-1][len-
   1]=='a' || words[i-1][len-1]=='e' || words[i-1][len-1]=='i' || words[i-1]
   [len-1]=='o' || words[i-1][len-1]=='u')) cnt[i]++;
11     }
12     int* result=(int*)malloc(queriesSize*sizeof(int));
13     for (int i=0;i<queriesSize;i++) {//注意是哪个size
14         result[i]=cnt[queries[i][1]+1]-cnt[queries[i][0]];
15     }
16     *returnSize=queriesSize;
17     return result;
18 }
```

1310范围内数字位异或结果

C

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* xorQueries(int* arr, int arrSize, int** queries, int queriesSize,
int* queriesColSize, int* returnSize) {
    *returnSize=queriesSize;
    int xors[arrSize+1];
    int* result=(int*)malloc(sizeof(int)*queriesSize);
    xors[0]=0;
    for (int i=1;i<=arrSize;i++) xors[i]=xors[i-1]^arr[i-1];
    for (int i=0;i<queriesSize;i++) {
        result[i]=xors[queries[i][1]+1]^xors[queriesSize[i][0]];
    }
    return reuslt;
}
```

560和为定值的子序列数量

C

```c
// 暴力美学：20行C代码
int subarraySum(int *nums, int numsSize, int k) {
    int count = 0;
    // 弄个大数组做个暴力的Hash表，大概4*20M*2=160M。用calloc初始化为全零。
    int *maps = (int *)calloc(1001 * 20001 * 2, sizeof(int));
    // 前缀和可能是负数，把指针放到中间位置
    int *map = maps + 1001 * 20001 * 1;
    // 补一个最前面的前缀和，用了下标0
    int sum = 0;
    map[sum]++;
    // 下标从1开始，注意=
    for (int i = 1; i <= numsSize; i++) {
        // 注意-1
        sum += nums[i - 1];
        if (map[sum - k] > 0) count += map[sum - k];
        map[sum]++;
    }
    free(maps);
    return count;
}
```

452用最少的箭引爆所有气球！！！

C

```c
int cmp(const void* a, const void* b) {
    int end_a = (*(int**)a)[1], end_b = (*(int**)b)[1];
    return (end_a > end_b) - (end_a < end_b);
}//按照右端点的位置排序
int findMinArrowShots(int** points, int pointsSize, int* pointsColSize) {
    qsort(points, pointsSize, sizeof(int*), cmp); // 根据区间右端值排序
    int ans = 1;
    int pre = points[0][1];
    for (int i = 1; i < pointsSize; i++) {
        if (points[i][0] > pre) { // 上一个放的点在区间左边
            ans++;
            pre = points[i][1]; // 在区间的最右边放一个点
        }
    }
    return ans;
}
```

3905多源图像渲染

C

```c
/**
 * Return an array of arrays of size *returnSize.
 * The sizes of the arrays are returned as *returnColumnSizes array.
 * Note: Both returned array and *columnSizes array must be malloced,
assume caller calls free().
 */
int** colorGrid(int n, int m, int** sources, int sourcesSize, int*
sourcesColSize, int* returnSize, int** returnColumnSizes) {
    int** col=(int**)malloc(n*sizeof(int*));
    for (int i=0;i<n;i++) col[i]=(int*)malloc(m*sizeof(int));
    for (int i=0;i<n;i++) {
        for (int j=0;j<m;j++) col[i][j]=0;
    }
    *returnSize=n;
    *returnColumnSizes=(int*)malloc(n*sizeof(int));
    for (int i=0;i<n;i++) (*returnColumnSizes)[i]=m;
    for (int i=0;i<sourcesSize;i++) col[sources[i][0]][sources[i]
[1]]=sources[i][2];
    for (int i=0;i<n;i++) {
        for (int j=0;j<m;j++) {
            int num=0,pos=-1,dis=INT_MAX;
            for (int k=0;k<sourcesSize;k++) {
```

```
                int temp=abs(sources[k][0]-i)+abs(sources[k][1]-j);
                if (temp<dis || temp==dis && sources[k][2]>num) {
                    pos=k;
                    dis=temp;
                    num=sources[k][2];
                }
            }
            col[i][j]=num;
        }
    }
    return col;
}
//该程序运行时间超时
```

C

```
/**
 * Return an array of arrays of size *returnSize.
 * The sizes of the arrays are returned as *returnColumnSizes array.
 * Note: Both returned array and *columnSizes array must be malloced, assume caller calls free().
 */
int cmp(const void *a, const void *b) {
    int *pa = *(int **)a;
    int *pb = *(int **)b;
    return pa[2] - pb[2];   // 按颜色升序
}

int** colorGrid(int n, int m, int** sources, int sourcesSize, int* sourcesColSize, int* returnSize, int** returnColumnSizes) {
    // 分配并初始化返回网格
    int** col = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        col[i] = (int*)malloc(m * sizeof(int));
        for (int j = 0; j < m; j++) {
            col[i][j] = 0;
        }
    }

    *returnSize = n;
    *returnColumnSizes = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) (*returnColumnSizes)[i] = m;

    // 按颜色升序排序（小颜色在前，大颜色在后）
    qsort(sources, sourcesSize, sizeof(int*), cmp);

```

```c
    // 先将所有源点颜色写入网格（后写的会覆盖先写的，即大颜色保留）
    for (int i = 0; i < sourcesSize; i++) {
        int r = sources[i][0];
        int c = sources[i][1];
        int color = sources[i][2];
        col[r][c] = color;
    }

    // 分配队列（最多入队 m*n 个元素，索引 0 ~ m*n-1）
    int** que = (int**)malloc(m * n * sizeof(int*));
    for (int i = 0; i < m * n; i++) {
        que[i] = (int*)malloc(3 * sizeof(int));
    }

    int cnt = 0, total = sourcesSize;
    // 源点倒序入队（颜色大的先入队，保证大颜色优先扩散）
    for (cnt = 0; cnt < sourcesSize; cnt++) {
        int idx = sourcesSize - 1 - cnt;
        que[cnt][0] = sources[idx][0];
        que[cnt][1] = sources[idx][1];
        que[cnt][2] = sources[idx][2];
    }

    // 多源 BFS
    for (cnt = 0; cnt < n * m; cnt++) {
        if (cnt >= total) break;   // 队列已空，停止处理

        int r = que[cnt][0];
        int c = que[cnt][1];
        int color = que[cnt][2];

        // 若该单元格尚未最终着色（值为 -1 或 0），赋予颜色
        if (col[r][c] == -1 || col[r][c] == 0) {
            col[r][c] = color;
        }

        // 向四个方向扩散
        // 上
        if (r - 1 >= 0 && col[r - 1][c] == 0) {
            col[r - 1][c] = -1;                 // 标记为“已入队待着色”
            que[total][0] = r - 1;
            que[total][1] = c;
            que[total][2] = color;
            if (total < m * n) total++;         // 防止越界（但实际 total 不会超过 m*n）
        }
        // 下
        if (r + 1 < n && col[r + 1][c] == 0) {
```

```
            col[r + 1][c] = -1;
            que[total][0] = r + 1;
            que[total][1] = c;
            que[total][2] = color;
            if (total < m * n) total++;
        }
        // 左
        if (c - 1 >= 0 && col[r][c - 1] == 0) {
            col[r][c - 1] = -1;
            que[total][0] = r;
            que[total][1] = c - 1;
            que[total][2] = color;
            if (total < m * n) total++;
        }
        // 右
        if (c + 1 < m && col[r][c + 1] == 0) {
            col[r][c + 1] = -1;
            que[total][0] = r;
            que[total][1] = c + 1;
            que[total][2] = color;
            if (total < m * n) total++;
        }
    }

    // 释放队列内存
    for (int i = 0; i < m * n; i++) free(que[i]);
    free(que);

    return col;
}
```

1524最长的超赞字符串

C

```
//给你一个字符串 s 。请返回 s 中最长的 超赞子字符串 的长度。「超赞子字符串」需满足满足下述两个条件：该字符串是 s 的一个非空子字符串 进行任意次数的字符交换后，该字符串可以变成一个回文字符串
int longestAwesome(char* s) {
    int len = strlen(s), result = 0;
    int cnt[len + 1][10], cal[len + 1], fin[1024];
    // 初始化 fin 数组为 -1，表示状态从未出现过
    for (int i = 0; i < 1024; i++) fin[i] = -1;
    fin[0] = 0;   // 空前缀状态为 0，位置为 0
    // 初始化 cnt[0] 和 cal[0]
    for (int j = 0; j < 10; j++) cnt[0][j] = 0;
```

```
    cal[0] = 0;
    // 计算每个位置的前缀奇偶状态
    for (int i = 1; i <= len; i++) {
        // 继承上一个位置的状态
        for (int j = 0; j < 10; j++) cnt[i][j] = cnt[i - 1][j];
        // 翻转当前数字对应的奇偶位
        cnt[i][s[i - 1] - '0'] ^= 1;
        // 将奇偶状态编码为二进制整数
        cal[i] = 0;
        for (int j = 0; j < 10; j++)
            cal[i] += cnt[i][j] * (1 << j);   // 使用位移替代 pow
        // 检查全偶状态（允许 0 个奇数位）
        if (fin[cal[i]] != -1) {
            int len_sub = i - fin[cal[i]];
            if (len_sub > result) result = len_sub;
        }
        // 检查允许 1 个奇数位的情况（翻转某一位）
        for (int j = 0; j < 10; j++) {
            int target = cal[i] ^ (1 << j);
            if (fin[target] != -1) {
                int len_sub = i - fin[target];
                if (len_sub > result) result = len_sub;
            }
        }
        // 如果是首次遇到该状态，记录位置
        if (fin[cal[i]] == -1) fin[cal[i]] = i;
    }
    return result;
}
/**思路：首先用cnt数组记录截止到某一位置，0~9各个数字的出现次数，并对2取模，因为如果能够被2整除即可对称
 * 而后将每个位置的出现次数视为二进制数，将对应的十进制数转化入cal数组
 * 最后在fin数组中记录每个十进制数出现的第一个位置
 * 对于fin数组，我们反转某一位，得到所有的可能与该处恰好只多一个数字的情况并进行检查，根据情况更新
 */
```

1371每个原因均出现偶数次的最长子串

C

```
int findTheLongestSubstring(char* s) {
    int len=strlen(s),result=0;
    int cal[len+1][5],cnt[len+1],fin[32];
    for (int i=0;i<32;i++) fin[i]=i==0?0:-1;
    for (int i=0;i<=len;i++) for (int j=0;j<5;j++) cal[i][j]=0,cnt[i]=0;
```

```
    for (int i=1;i<=len;i++) {
        for (int j=0;j<5;j++) cal[i][j]=cal[i-1][j];//注意要先继承在操作
        if (s[i-1]=='a') cal[i][0]=(cal[i][0]+1)%2;//注意是i还是i-1
        else if (s[i-1]=='e') cal[i][1]=(cal[i][1]+1)%2;
        else if (s[i-1]=='i') cal[i][2]=(cal[i][2]+1)%2;
        else if (s[i-1]=='o') cal[i][3]=(cal[i][3]+1)%2;
        else if (s[i-1]=='u') cal[i][4]=(cal[i][4]+1)%2;
    }
    for (int i=0;i<=len;i++) {
        for (int j=0;j<5;j++) cnt[i]+=cal[i][j]*pow(2,j);
        if (fin[cnt[i]]==-1) fin[cnt[i]]=i;//注意0是一个合法的位置，则有可能更新后再次修改，故使用-1
        else result=i-fin[cnt[i]]>result?i-fin[cnt[i]]:result;
    }
    return result;
}
```